

antd 测试库迁移的那些事儿

antd 测试库迁移的那些事儿

大家好，我是 @li-jia-nan。也是前几个月新加入 antd 的 Collaborator，有幸作为 Collaborators 之一，我开发了 **FloatButton** 组件和 **QRCode** 组件，以及一些其它维护工作，下面分享一下 antd 测试库迁移的那些事儿～

引言

在 antd@4.x 中，使用 **enzyme** 作为测试框架，然而由于 enzyme 缺乏维护，到了 React 18 时代已经很难支持。也因此不得不开始为 antd 开启漫长的 **@testing-lib** 迁移之路。

在迁移过程中，我承担了大概 antd 四分之一的工作量，这里主要记录一下迁移过程中遇到的问题。

感谢在此期间 @zombieJ @MadCcc @miracles1919 提供的帮助。

图片说明：image

图片说明：image

图片说明：image

起步

在迁移之前，我们需要先搞清楚迁移的目的是什么。在 enzyme 中，大多数场景是测试了组件中的状态是否正确，或者 class 上的静态属性是否正常被赋值，这其实是不合理的，因为我们更重要的是需要关心“功能”是否正常，而非“属性”是否正确，因为源代码对使用者来说是黑盒，用户只关心组件是否正确。

基上，测试用例应该基于“行为”来编写，而非“实现”来编写（这也是 testing-library 的目标）。在这个原则上，会发现有几个用例是多余的（因为在实际代码中不会单独触发某些函数），将其删除也并没有影响到 test coverage。

当然了，这只是放弃 enzyme 的其中一个原因。更重要的是它缺乏维护，并且不支持 React 18 了。

迁移

一、渲染：

enzyme 支持三种方式的渲染：

- shallow: 浅渲染，是对官方的 Shallow Renderer 的封装。将组件渲染成虚拟 DOM 对象，通过 Shallow Render 得到的组件不会有断言到子组件的部分，并且可以使用 jQuery 的方式访问组件的信息。
- render: 静态渲染，它将 React 组件渲染成静态的 HTML 字符串，然后解析这段字符串，并返回一个实例对象，可以用来分析组件的 html 结构。
- mount: 完全渲染，它将组件渲染加载成一个真实的 DOM 节点，用来测试 DOM API 的交互和组件的生命周期，用到了 jsdom 来模拟浏览器环境。

为了贴近浏览器现实场景，antd@4.x 选用 mount 来进行渲染，而在 @testing-library 中对应的则是 render 方法：

```
-- import { mount } from 'enzyme';
++ import { render } from '@testing-library/react';

-- const wrapper = mount(
++ const { container } = render(

,
);
```

二、交互 & 事件

enzyme 提供了 simulate(event) 方法来模拟事件触发和用户交互，event 为事件名称，而在 @testing-library 中对应的则是 fireEvent 方法：

```
++ import { fireEvent } from '@testing-library/react';

-- wrapper.find('.ant-handle').simulate('click');
++ fireEvent.click(container.querySelector('.ant-handle'));
```

三、DOM 元素

在 enzyme 中，提供了一些内置的 api 来操作 dom，或者查找组件：

- instance(): 返回测试组件的实例

- `at(index)`: 返回一个渲染过的对象
- `text()`: 返回当前组件的文本内容
- `html()`: 返回当前组件的 HTML 代码形式
- `props()`: 返回组件的所有属性
- `prop(key)`: 返回组件的指定属性
- `state()`: 返回组件的状态
- `setState(nextState)`: 设置组件的状态
- `setProps(nextProps)`: 设置组件的属性
- `find(selector)`: 根据选择器查找节点, `selector` 可以是 CSS 中的选择器, 也可以是组件的构造函数, 以及组件的 `displayName` 等

在 `testing-library` 中, 没有提供这些 api (正如上面提到过的 - `testing-library` 更加注重行为上的测试), 所以需要换成原生的 dom 操作:

```
expect(ref.current.getPopupDomNode()).toBe(null);
-- popover.find('span').simulate('click');
-- expect(popover.find('Trigger PopupInner').props().visible).toBeTruthy();

++ expect(container.querySelector('.ant-popover-inner-content')).toBeFalsy();
++ fireEvent.click(popover.container.querySelector('span'));
++ expect(container.querySelector('.ant-popover-inner-content')).toBeTruthy();
```

四、兼容性测试

在大版本升级的同时, 废弃了部分组件, 但是并没有在 `antd` 中移除, 比如 `BackTop` 组件, 需要在组件中加入 `warning` 以保证兼容性, 所以还需要对 `warning` 编写专门的单元测试:

```
describe('BackTop', () => {
++  it('should console Error', () => {
++    const errSpy = jest.spyOn(console, 'error').mockImplementation(() => {});
++    render();
++    expect(errSpy).toHaveBeenCalledWith(
++      'Warning: [antd: BackTop] `BackTop` is deprecated. Please use `FloatButton.BackTop` instead.',
++    );
++    errSpy.mockRestore();
++  });
});
```

Diff 之谜

在转换过程中，发现了一个神奇的现象，有些情况下，同样的 case 生成的 DOM 快照会不一样，也因此开始探索 React 18 到底变化了什么：

过去 enzyme 的 snapshot 对比是通过 enzyme-to-json 插件将 enzyme object 转换成序列化对象：

```
//jest.config.js
module.exports = {
  // ...
  snapshotSerializers: ['enzyme-to-json/serializer'],
};
```

到了 @testing-library/react 则直接通过调用 render 产生 dom 元素，然后对 dom 进行对比：

```
-- import { mount } from 'enzyme';
++ import { render } from '@testing-library/react';

describe('xxx', () => {
  it('yyy', () => {
--   const wrapper = mount();
++   const { container } = render();
--   expect(wrapper.render()).toMatchSnapshot();
++   expect(container.firstChild).toMatchSnapshot();
  });
});
```

有趣的是，在一些测试用例中。它会挂掉，区别在于 React 18 有时候会少一些空行：

```
--
  Hello World
```

通过测试 dom 的 innerHTML 后发现，17 和 18 是一样的。所以在遇到问题之初，我们只是将测试用例简单的改成比较 innerHTML：

```
expect(container.querySelector('.className').innerHTML).toMatchSnapshot();
```

但是，随着迁移变多，会逐渐发现这种情况不断发生。比较 innerHTML 也不是长久之计。于是开始探索为什么会出现这种情况。

pretty-format

pretty-format 是一个很有意思的库，它可以将任意对象转换成字符串。它的一个用途就是用于 jest 的 snapshot 对比。它的一个特点是可以自定义转换规则。

jest 中对比 snapshot 会先做一步 format，对于原生 dom、object 等常见对象。它已经内置了一套 plugins 用以做格式化转换：

```
Hello  
World
```

↓

```
Hello  
World
```

出现多余空格第一反应就是是否是因为 17 & 18 引入的 @testing-lib/react 版本不同，导致影响了 jest 依赖的 pretty-format 版本，经过检查都是一致的：

```
{  
  "devDependencies": {  
    "pretty-format": "^29.0.0",  
    "@testing-library/react": "^13.0.0"  
  }  
}
```

这个判断不对后，那就是另一种情况。dom 中存在空元素，使得 pretty-format 可以感知，但是本身却不影响 innerHTML，于是就写了一个简单的 test case：

```
const holder = document.createElement('div');  
holder.append("");  
holder.append(document.createElement('a'));  
expect(holder).toMatchSnapshot();  
console.log(holder.innerHTML);
```

得到以下输出：

```
// snapshot  
exports[`debug exports modules correctly 1`] = `  
  
`;  
`;
```

```
// console.log
```

和设想的一致，那么就很简单了。那么大概率就是 React 18 的 render 会忽略空元素。我们做一个简单的实验：

```
const App: React.FC = () => {  
  const holderRef = useRef(null);  
  useEffect(() => {  
    console.log(holderRef.current?.childNodes);  
  }, []);  
  return (  
  
    {version}  
  
  );  
};
```

果不其然：

- React 17: NodeList(2) [text, p]; React 18: NodeList [p]

检查一下 Fiber 节点信息，可以发现 React 17 会把空元素也作为 Fiber 节点，而 React 18 则会忽略空元素：

React 17:

图片说明: image

React 18:

图片说明: image

按图索骥就能找到相关 PR：

- <https://github.com/facebook/react/pull/22807>

图片说明: WX20230319-145539@2x

一个解法

antd 需要对 React16、17、18 都进行测试，如果 snapshot 不可行会造成太大成本。所以我们需要对 jest 进行改造。enzyme-to-json 则给了我灵感，我们可以修改 snapshot 生成逻辑来抹平 React 不同版本之间的 diff：

```
expect.addSnapshotSerializer({  
  // 判断一下是否是 dom 元素，如果是的就走我们自己的序列化逻辑  
  // 代码简化过，真实判断需要更多逻辑，可以参考 antd 的 setupAfterEnv.ts  
  test: (element) => element instanceof HTMLElement,  
  // ...  
});
```

然后接入 pretty-format，添加自己的逻辑：

```
const htmlContent = format(element, {  
  plugins: [plugins.DOMCollection, plugins.DOMElement],  
});
```

```
expect.addSnapshotSerializer({  
  test: '//*[...]',  
  print: (element) => {  
    const filtered = htmlContent  
      .split(/\n\r+/)  
      .filter((line) => line.trim())  
      .map((line) => line.replace(/\s+$/, ''))  
      .join('\n');  
    return filtered;  
  },  
});
```

收工

以上，是 antd 测试框架迁移时遇到的一些问题，希望对于需要迁移或者尚未开始编写测试用例的同学提供帮助。也欢迎大家加入 antd 社区，共同为开源奉献自己的力量。